Worst-Case Time Analysis of Key Agreement Protocols
in 10BASE-T1S Automotive Networks

Teodora Ljubevska[1], Alexander Zeh[2], Donjete Elshani Rama[2], and Ken Tindell[3]

[1] HENSOLDT
teodora.ljubevska@hensoldt.net,
[2] Infineon Technologies AG, Automotive Division, Research & Development
85579 München, Germany
{Alexander.Zeh, Donjete.Elshani}@infineon.com
[3] JK Energy
ken.tindell@jkenergy.com

Abstract. With the rise of in-vehicle and car-to-x communication systems, ensuring robust security in automotive networks is becoming increasingly vital. As the industry shifts toward Ethernet-based architectures, the IEEE 802.1AE MACsec standard is gaining prominence as a critical security solution for future in-vehicle networks (IVNs). MACsec utilizes the MACsec Key Agreement Protocol (MKA), defined in the IEEE 802.1X standard, to establish secure encryption keys for data transmission. However, when applied to 10BASE-T1S Ethernet networks with multidrop topologies, MKA encounters a significant challenge known as the real-time paradox. This paradox arises from the competing demands of prioritizing key agreement messages and real-time control data, which conflict with each other. Infineon addresses this challenge with its innovative In-Line Key Agreement (IKA) protocol. By embedding key agreement information directly within a standard data frame, IKA effectively resolves the real-time paradox and enhances network performance. This paper establishes a theoretical worst-case delay bound for key agreement in multidrop 10BASE-T1S IVNs with more than two nodes, using Network Calculus techniques. The analysis compares the MKA and IKA protocols in terms of performance. For a startup scenario involving a 16-node network with a 50 bytes MPDU size, the MKA protocol exhibits a worst-case delay that is 1080 % higher than that of IKA. As the MPDU size increases to 1486 bytes, this performance gap narrows significantly, reducing the delay difference to just 6.6 %.

Keywords: Automotive Ethernet · Key Agreement Protocols · MKA · Real-Time · Worst-Case Time Analysis.

1 Introduction

The automotive E/E architecture is moving from domain-based to zonal architecture. With this shift, zones of sensors and actuators require low-latency

communication and high-bandwidth [4]. Automotive Ethernet is becoming the key technology for IVNs, the high speed Ethernet enabled its use in the backbone of the vehicle [12]. However, seeing the advantages of the Ethernet technology, which has been long established in the IT industry, the car makers want to further expand and use it where possible. This we see also with the new Ethernet standard 10BASE-T1S, with multidrop topology [6]. Considering that nearly 90% of in-vehicle nodes operate at 10 Mbits/s or lower, 10BASE-T1S is suited for various applications, including body control and other low-bandwidth functions within the vehicle network [3].

Apart from the PHY layer, the rich Ethernet stack also offers layers of security protocols. Following the concept of defense in depth [3], the data link layer provides the MACsec protocol, which is specified in IEEE 802.1AE [8]. MACsec protects and encrypts the link-to-link data exchanged by Electronic Control Units (ECUs) against security breaches [11].

Known from the IT domain, the corresponding protocol to establish Secure Associaton Keys (SAKs) for encrypting the frames is called MACsec Key Agreement (MKA), specified in IEEE 802.1X [7]. However, the real-time paradox arises when MKA is deployed in a multidrop environment. The paradox describes the conflict between the urgency of key establishment messages and real-time control messages. When prioritizing key establishment, some control messages may experience frame loss. On the other hand, when prioritizing control traffic, the key establishment is prolonged, and secure communication starts late.

Our paper analyzes MKA and IKA worst-case time until the final key agreement in an IVN with more than two nodes. We use state-of-the-art Network Calculus (NC), an academic framework that analyzes performance guarantees in computer networks [9].

Section 2 gives the essential requirements on MKA in the automotive domain. We recall essential mechanisms of PLCA, and the required basics of NC in Section 4. IKA is introduced in Section 3. The model for the worst-case time analysis is described in Section 5 and the final results are presented in Section 6. We conclude in Section 7.

2 State of the Art

Automotive applications necessitate modifications to MKA (MACsec Key Agreement) to meet the stringent performance and reliability demands of vehicle environments. Standard MKA implementations typically require 3 to 8 seconds to initiate MACsec-protected traffic, which is inadequate for automotive scenarios [19]. To address this, the OPEN Alliance has introduced the Automotive MKA Specification [15], which sets specific requirements, including a maximum allowable startup time of 30 milliseconds for key agreement processes.

The specification also highlights certain optimizations, such as the use of pre-shared Connectivity Association Keys (CAKs) and the activation of Extended Packet Numbers (XPN), to reduce rekeying overhead [15]. Looking ahead, future

development will aim to enhance MKA for multidrop configurations, a feature that remains unstandardized as of April 2025 [16].

3 The In-Line Key Agreement (IKA) Protocol

The IKA protocol, introduced in 2023, provides an alternative to MKA by eliminating the need for a centralized Key Server. Unlike MKA, where the Key Server coordinates key distribution, IKA derives session keys locally at each node using a Key Derivation Function that takes, among other inputs, a Key Number (KN) transmitted in-line within an IKA PDU.

The IKA protocol enables MACsec security entities (SecY instances) to establish and use session keys during communication. IKA operates on top of MACsec. It ensures the synchronization of packet numbers, protects against replay attacks, and generates session keys. The necessary key agreement data is transmitted directly within Ethernet frames. IKA utilizes the MACsec Layer Management Interface (LMI) to manage SecY functions. Furthermore, IKA for Ethernet exploits the EtherType chaining property of the protocol. Figure 1 shows the encapsulation of user data (i.e., an MSDU) by a MACsec PDU (MPDU), itself encapsulated by an IKA PDU (IPDU). The MSDU may be a length code with application data or may itself be further PDUs. The size of an IKA PCI is 14 bytes. The IKA KaY instance performs key agreement and PN synchronization on every MACsec message. There are no separate key agreement messages (and so the bandwidth for key agreement is near zero).

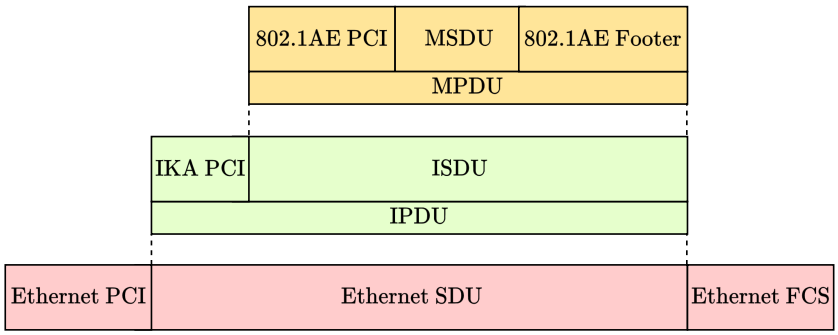

Fig. 1. Encapsulation of MACsec MPDU with an IKA SDU (ISDU) resulting in an IKA PDU (IPDU) resp. an Ethernet SDU.

4 PLCA, and Network Calculus

This section presents three fundamental concepts we combine for our model in Section 5: PLCA in the PHY layer, the key agreement processes in MKA and IKA, and Network Calculus.

The Physical Layer Collision Avoidance (PLCA) is defined in IEEE 802.3 for half-duplex multidrop networks, supporting up to 8 nodes over a 25-meter trunk [5, p. 5884]. Each node is assigned a unique ID $i \in [0, m]$, shown in the Figure 2. Transmission opportunities (TOs) are granted sequentially based on this ID in a round-robin manner, ensuring fairness. The node with ID 0 is called the coordinator. It starts each cycle by sending a BEACON signal. Afterward, every node i gets its turn to transmit packets during its own TO. If a node has data, it sends a COMMIT signal and the Ethernet frame. If not, it waits for a short SILENCE interval before the TO moves to the next node. The maximum bus cycle occurs if each node transmits packets of maximum size [13].

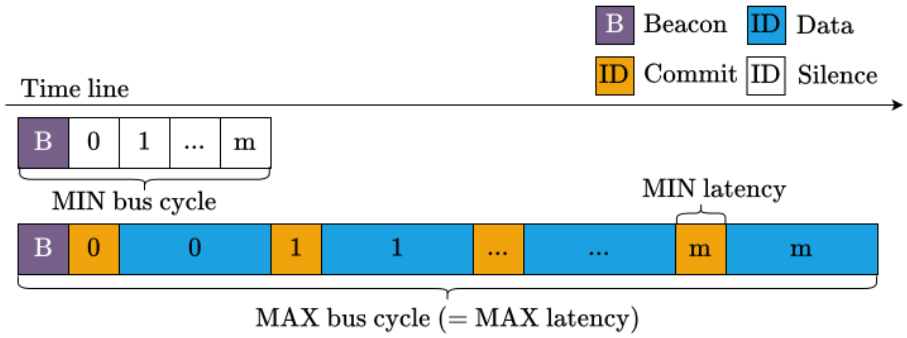

Fig. 2. Transmission sequence of $m + 1$ nodes using PLCA. A bus cycle begins with a BEACON sent by node 0 to initialize the Transmission Opportunity (TO) counter. The minimum bus cycle occurs when all nodes remain silent, while the maximum bus cycle occurs when all nodes transmit their maximum data packet. Reworked from [13].

In addition, PLCA offers two transmission modes: normal mode and burst mode. Our model uses the normal mode where a node i can send only a single packet within its TO [14]. To explore theoretical scalability, our analysis extends the number of nodes beyond the 8-node limit to 16 nodes to push network conditions. This configuration is not intended for real-world implementation but rather to understand protocol limitations. For such higher node counts, alternative architectures, such as switched Ethernet topologies, would be required to meet practical performance demands. We also follow the BEACON and Interpacket Gap (IPG) durations as specified in the Open Alliance specification of 10B-T1S [15].

NC is a system theory for computer networks. In order to derive worst-case deterministic bounds, a model of data flows and the network is required. The idea is to derive a lower bound of the offered service by the network and an upper bound of the data flow. Arrival curves model the flows, and service curves model the network with its nodes [9].

Definition 1 (Arrival Curve): A wide-sense increasing arrival curve $\alpha(t)$ for $t \geq 0$ bounds the cumulative amount of data arriving at a network node over time. The positive function $R(t)$ counts the data flow (total bits received by time t) during the interval $[0, t]$. A flow R is considered to be constrained by α if and only if for every $\forall s \leq t$, the following condition (see [14]) holds:

$$R(t) - R(s) \leq \alpha(t - s).$$

We use the common token-bucket arrival where $r^i = {}^{l^i}/_{T^i}$ represents the maximum long-term arrival rate of data, where $i \in [0, m]$ denotes the node ID. Hence, we have $m+1$ nodes in the network. The burst b^i of the flow indicates the maximum volume of data that can arrive. Consequently, a flow with a minimum inter-packet arrival time T^i and a maximum packet length l^i at node i is constrained by a leaky-bucket arrival curve: [14]

$$\alpha^i(t) = \begin{cases} {}^{l^i}/_{T^i} \cdot t + b^i, & \text{if } t > 0 \\ 0, & \text{otherwise.} \end{cases} \tag{1}$$

Definition 2 (Service Curve): A system S offers a service curve $\beta(t)$ if and only if β is wide-sense increasing with $\beta(0) = 0$ and for $t > 0$. We use the rate-latency curve, defined as [14]

$$\beta_{R,T}(t) = R \cdot [t - T]^+. \tag{2}$$

R represents the minimum guaranteed long-term service rate the system can offer the flow. T is the latency the flow will experience when crossing the system before service begins [14]. The term $[x]^+ = \max(0, x)$ ensures that service starts only after the initial latency period. For a rate-latency service curve $\beta_{R,T}$ and a leaky bucket arrival curve, the network is stable if and only if $r < R$ [1].

In a PLCA-based multidrop network, the latency corresponds to the maximum waiting time before a node can access the medium, and the rate represents the effective service rate assigned to the node [14]. In our model, each node with the ID $i \in [0, m]$ receives at least the PLCA service rate

$$R^i = \frac{q^{i,min}}{q^{i,min} + Q^{i,max}} \cdot C, \tag{3}$$

where $q^{i,min}$ represents the guaranteed transmission quantum (in bits) allocated to the node i in a PLCA cycle. It includes the minimum packet length $l^{i,min}$. The maximum cross-traffic quantum of service for transmission allocated to all other node is $Q^{i,max}$. It includes the BEACON of 20 bits and the sum of the other nodes' maximum packet size $l^{i>0,max}$ (see [14]). A node's waiting time for its TO is influenced by $Q^{i,max}$. The latency T^i at node i to access the multidrop is upper bounded by [14, Eq. (9)]

$$T^i = \frac{Q^{i,max}}{C}. \tag{4}$$

Theorem 1 (PLCA Service Curve [14]**)** *Let R and T be the rate and latency parameters defined in* (3) *and* (4)*, respectively. By substituting these into the rate-latency service curve as defined in Definition 2, the service curve offered by the PLCA mechanism to node i is given by:*

$$\beta^i_{PLCA}(t) = \frac{q^{i,min}}{q^{i,min} + Q^{i,max}} \cdot C \cdot \left[t - \frac{Q^{i,max}}{C} \right]^+ . \tag{5}$$

A proof of Thm. 1 is available in [14, p. 6-7].

Theorem 2 (Non-Preemptive Priority Node [10, Section 6.2]**)** *Consider a constant bit rate server, with rate C, serving two flows, high-priority (H) and low-priority (L), with non-preemptive priority given to H. The server guarantees a strict service curve β to the aggregate of the flows. Then flow H is guaranteed a rate-latency service curve with rate C and latency l^L_{max}/C where l^L_{max} is the maximum packet size of L. Assuming the high priority flow follows a leaky-bucket arrival curve with $r < C$ and burst b, then the low priority flow is guaranteed a service rate of $C - r$ and latency $b/C-r$.*

For our model, this translates into the service curve β^i_H for a high-priority flow H of node i

$$\beta^i_H(t) = R^i \cdot \left[t - \frac{Q^{i,max}}{C} - \frac{l^i_L}{C} \right]^+ , \tag{6}$$

where l^i_L denotes the maximum packet length of flow L. Suppose a high-priority packet arrives while a low-priority packet is already transmitted. In that case, the high-priority packet must wait l^i_L/C until the current low-priority packet transmission finishes. Hence, the total access latency is increased.
The service curve for low-priority traffic L, using the PLCA service rate, is as follows:

$$\beta^i_L(t) = (R^i - r^i_H) \cdot \left[t - \frac{Q^{i,max}}{C} - \frac{b_H}{R^i - r^i_H} \right]^+ . \tag{7}$$

The arrival rate of flow H is denoted by r^i_H and b_H denotes the high-priority arrival burst.

Theorem 3 (Delay Bound [14, Thm. 2]**)** *Consider a flow at node i with an arrival curve α crossing a server with a service curve β. According to NC, the worst-case delay $D^i(t)$ experienced by the flow is bounded by the maximum horizontal distance between α and β for all $t \in \mathbb{R}^+$ such that*

$$D^i(t) \leq \inf\{d \geq 0 \mid (\alpha^i \oslash \beta^i)(-d) \leq 0\}. \tag{8}$$

The operator $\oslash$ denotes the min-plus deconvolution. Hence, for a low-priority flow at node i, we use the service curve in (7), and we define the low-priority arrival curve α^i_L using the low-priority packet size l^i_L.

5 Worst-Case Modeling of MKA and IKA

We use the Single PLCA WRR Server modeling according to [14]. The input arrival curves of the MAC layer separated into two priority queues are directly connected to a single PLCA server in the PHY layer. Our NC model results in a conservative delay bound given that the two low- and high-priority data flows compete for multidrop access (the PLCA service curve) in the MKA case. Since IPDUs are not separate packets, unlike MKDPUs, we use only one arrival curve in the IKA model.

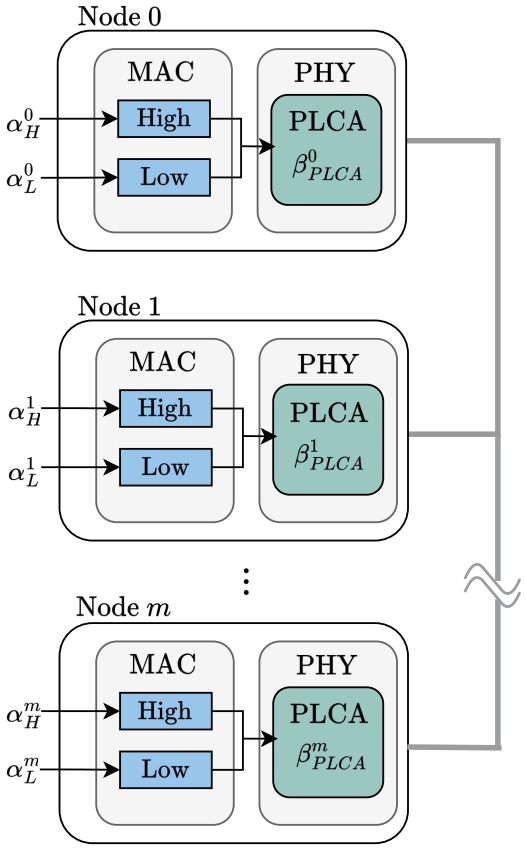

Fig. 3. Example of a PLCA multidrop network with $m + 1$ nodes and incoming low and high priority traffic. Light-blue rectangles in the MAC layer depict the priority queues, and darker blue rectangles in the PHY layer depict the PLCA server with the service curve β_{PLCA}^i. Reworked from [14].

For our model, we define the following assumptions:

1. All nodes $i \in [0, m]$ are in the same Connectivity Association (CA). The head node ($i = 0$) broadcasts. The rest of the nodes transmit unicast to the head node.
2. Nodes become sequentially operational during the startup phase.
3. We do not consider the propagation delay, jitter, or packet loss after transmission.
4. We use PLCA in normal mode.
5. CAK and SAK are 128 bits long.

Specific to MKA:

6. We assume node discovery happened and examine the key distribution with a fixed Key Server role (node 0).
7. There are two priority arrival flows, MACsec PDUS (MPDUs) and MKPDUs, competing for the PLCA service. We use non-preemptive scheduling.
8. MKPDUs are transmitted faster than the Hello Timer of 2s, specified in the Std. [7]. This corresponds to the requirement AutoMKA-03004 specified by Open Alliance [15].

Specific to IKA:

9. There is only one arrival flow of IPDUs taking the full PLCA service.

The key agreement is considered done when the head node ($i = 0$) sends its last key agreement message. In MKA, this corresponds to the last distributed SAK by the Key Server when all m nodes joined. In IKA, it refers to the last KN synchronization message from the head node to all other nodes. We use Thm. 3 to determine the worst-case delay per key agreement message at node 0, defined as $D^0_{\text{MKA/IKA}}$. The total worst-case time T_{WC} at startup denotes the timespan between the first key agreement message and the head node's last sent message

$$T_{WC} = D^0_{\text{MKA/IKA}} \cdot n^0_{\text{MKA/IKA}} \tag{9}$$

where $n^0_{\text{MKA/IKA}}$ is the number of key agreement messages that node 0 broadcasts, according to Table 1. We define the worst-case behavior as if each key agreement message experiences the worst-case delay $D^0_{\text{MKA/IKA}}$ before transmission.

We perform three experiments to evaluate the worst-case time to final key agreement T_{WC} of MKA versus IKA. In Experiment 1, the number of nodes is fixed at $m + 1 = 16$, while the MPDU size in bytes (B) is varied to observe its impact on T_{WC}. In Experiment 2, the MPDU size is fixed, and the number of nodes varies from 2 to 16. Experiment 3 demonstrates the impact of traffic prioritization by switching the priority levels of MKPDUs and MPDUs using the service curve formulations in Equations (6) and (7). Since there are no competing traffic classes in IKA, we use the full-service curve in Thm. 1 directly.

The assumptions for the experiments are:

1. The inter-arrival time parameters are set to $T^i_{\text{MKA}} = 500\text{ms}$, $T^i_{\text{MACsec}} = T^i_{\text{IKA}} = 73\text{ms}$ for all nodes $i \in [0, m]$ across all experiments.
2. In Experiments 2 and 3, the MKPDUs are high-priority.

6 Results

This section presents a comparative analysis of MKA and IKA regarding message count, packet size, and their impact on worst-case delay behavior across varying network conditions.According to IEEE 802.1X [7], the Key Server distributes a new SAK (with a different Key Number KN) when a node is added to its Live Peer List. This happens when the new node sends its first Hello message after it has received the Key Server's MI in a previous MKPDU to prove its liveness. In our case, the Key Server initiates the communication, see Diagram 7. In total, the KS broadcasts $m+1$ MKPDUs. The quadratic term $m+(m+1)\frac{m}{2}$ arises from the SAK confirmations that the Key server has to receive after the nodes $[1, m]$ have installed the key. In IKA, the SAK is derived implicitly at

		Total messages	**Unicast** $n^{i>0}_{\text{MKA/IKA}}$	**BC** $n^{0}_{\text{MKA/IKA}}$
MKA	Startup	$2m+1+(m+1)\frac{m}{2}$	$m+(m+1)\frac{m}{2}$	m+1
	Join	$(m+1)+2$	$m+1+1$	1
	Leave	$(m-1)+1$	$m-1$	1
IKA	Startup	$2m$	m	m
	Join	2	1	1
	Leave	0	0	0

Table 1. Comparison of key agreement messages in a multidrop network with $m+1$ nodes during startup, node join, and node leave scenarios. The table lists the total number of control messages divided into unicast and $n^{0}_{\text{MKA/IKA}}$ broadcast (BC) messages from node 0.

every node i. The Key Derivation Function (KDF) specified by NIST [2] using Counter Mode derives the SAK where the in-line transmitted Key Number (part of IKA PCI) serves as an input. Each node stores a set of known Key Numbers in its non-volatile memory. One of the circumstances when IKA shall re-key is when a node becomes online. This is shown in Figure 8 where Node 1 changes its KN to 1. The worst-case occurs if an IKA-enabled node receives an unknown KN smaller than its stored KN; see message (a) in Figure 8. In that case, the node will not decode the user data. This leads to m IPDUs from the head node to synchronize all joined nodes with its KN of 4. One of the parameter sets in MKA is the Live Peer List. It grows by 16 octets with every new live node. The theoretical maximum packet size comprises all parameter sets shown in IEEE Std 802.1X-2020 [7]. We take the practical maximum packet size, which leaves out the KMD, Announcement, XPN, and Distributed CAK Parameter Sets [[7], Section 11]. As a result, with a network size of 16 nodes, the maximum head node's l^{0}_{MKA} of 388 bytes is reached.

Unlike MKPDUs, the IPDU size does not depend on the number of live peers. This contrasts with MKA, where the MKDPU body length grows due to a larger

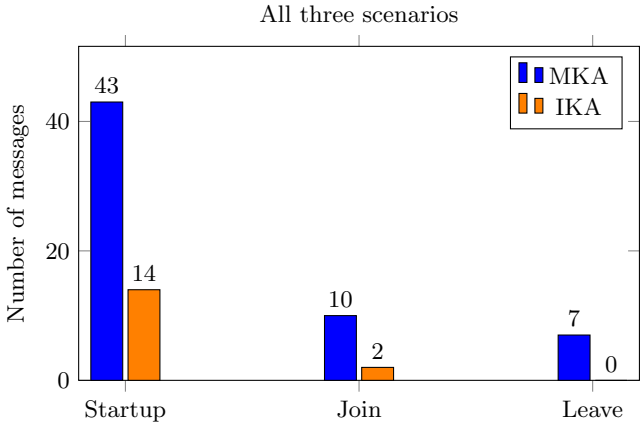

Fig. 4. Comparison of total control messages exchanged in MKA and IKA for a network with 8 nodes ($m = 7$). The figure shows the number of messages required during startup, node join, and node leave scenarios, based on the formulas in Table 1. It highlights the significantly lower control traffic in IKA compared to MKA.

Number of nodes $m+1$	$(l^0_{\text{MKA}}, l^i_{\text{MKA}})$ **in bytes** (**Head** $i = 0$, **Rest** $i \in [1, m]$)
1+1	(164, 132)
7+1	(260, 228)
15+1	(388, 356)

Table 2. The maximum MKPDU packet body lengths as tuples $(l^0_{\text{MKA}}, l^i_{\text{MKA}})$, where l^0_{MKA} is the length for the key server and l^i_{MKA} for all other nodes, depending on the total number of nodes $m + 1$ in the network.

Live Peer List. The 14-byte IKA PCI header is prepended to MPDU. To ensure the payload limit of IPDU $\leq$ 1500 bytes, the MPDU is constrained to [50, 1486] bytes.

On top of the MPDU (resp. IPDU), the MAC destination (6 bytes), MAC source (6 bytes), and Ethertype (2 bytes) are added, along with the Frame Check Sequence (FCS, 4 bytes) at the end. Furthermore, the Preamble (7 bytes) and Start Frame Delimiter (SFD, 1 byte) are added before the packet on the wire, while the Interpacket Gap (IPG, 12 bytes) follows each packet. All Ethernet parts must be included in the timing analysis at the wire level. [5] We account for the total overhead of 38 bytes. Figure 5 shows the results of Experiment 1, a U-shaped curve for MKA. This behavior arises from a trade-off between blocking frequency and blocking duration under non-preemptive scheduling:

– At 50 bytes MPDU size, the access latency is 4.8ms, and the PLCA service rate is 146kbps, resulting in T_{WC} of 448.6ms.

– Increasing the MPDU size to 356 bytes (equal to MKPDU size) improves the service rate to 625kbps with only a modest increase in access latency to 5.04ms, leading to the lowest delay: 168ms.
– Beyond 356 bytes, delays increase again. At 1486 bytes, access latency rises sharply to 19.5ms, and the service rate drops to 183kbps, yielding a worst-case delay of 610.2ms.

This represents a 63% reduction in delay from 50 bytes to 356 bytes, followed by a 263% increase from 356 bytes to 1486 bytes.

Under the IKA model, which has no prioritization, the PLCA service rate remains constant at 624kbps for all IPDU sizes at 16 nodes. According to Equation (4), the delay per IPDU message D^0_{IKA} increases linearly due to the growing access latency, from 1.27ms at 64 bytes to 18.5ms at 1500 bytes. Additionally,

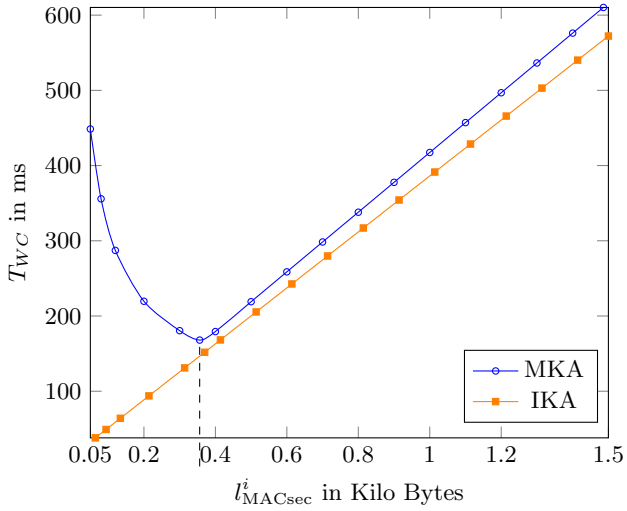

Fig. 5. Experiment 1: Worst-case time to final key agreement T_{WC} of high priority MKPDUs and IPDUs in dependence of an increasing MPDU size of all nodes l^i_{MACsec}. The network has a fixed size of 16 nodes ($m = 15$). T_{WC} under MKA shows a U-shape due to the trade-off between frequent blocking by small MPDUs (left region) and long blocking by large MPDUs (right region). In contrast, IKA, with only one packet type and constant service rate, shows a linear T_{WC} increase driven solely by access latency.

the results demonstrate that small MPDUs cause inefficient service due to a low guaranteed PLCA share (low $q^{0,min}$), which results in a low service rate. On the other hand, large MPDUs increase access latency. One reason is that the cross-traffic service $Q^{0,max}$ (determined by the MPDUs of other nodes) rises, and the other is that the head node's own low-priority MACsec packet blocks the TO longer (increased l^0_{MACsec}).

Overall, in Figure 5, at 16 nodes and 50 bytes MPDU size, the MKA protocol results in a 1080% higher T_{WC} than IKA due to repeated short blocking and low

service rate. At the largest MPDU size of 1486 bytes, this gap shrinks to only 6.6%, as access latencies and low service rates dominate both protocols.

In Figure 6 - Exp 2., the MPDU size is fixed per curve, and the number of nodes varies from 2 to 16. As the node count increases, the cross-traffic $Q^{0,max}$ at the head node increases, which leads to a longer PLCA cycle and higher access latency. For large 1486 bytes MPDUs, MKA and IKA worst-case time T_{WC} increases similarly with node count. Both protocols reach a similar access latency of 18.4ms for IKA and 19.4ms for MKA in a 16-node network. Despite the service rate at IKA being 3.4 times higher, the increased arrival rate of 1500 bytes every 73ms offsets the advantages of its higher service rate. In contrast, MKA has a much lower arrival rate for MKPDUs (every 500ms) and a lower service rate. As a result, the horizontal deviation between the arrival and service curve remains nearly identical in both protocols.

However, for small MPDUs of 50 bytes and 120 bytes, MKA scales significantly worse due to two compounding effects. On the one hand, the node count reduces the Key Server's service share while amplifying the blocking impact of frequent, small, low-priority frames. Our third Experiment 3 shows that the same U-shape

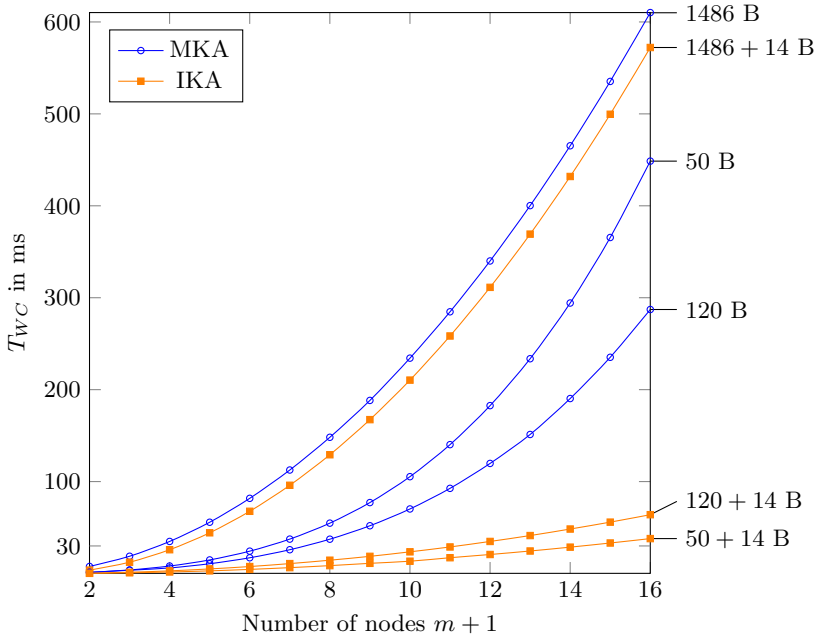

Fig. 6. Experiment 2: Worst-case time to final key agreement T_{WC} of high priority MKPDUs and IPDUs during the startup phase with a fixed MPDU and IPDU size (denoted on the right axis in bytes B) in dependence of the number of nodes $m + 1$.

observed in Experiment 1 (Figure 5) also appears when MKPDUs are assigned

low priority. Specifically, T_{WC} decreases from 0.556s to 0.256s, then grows to 15.98s, a relative increase of 6142% from the minimum at 356 bytes. This is due to repeated blocking by large, high-priority MPDUs. In contrast, for high-priority MKPDUs, the increase over the same range is just 23%. The impact of MKA priority is also reflected in the MACsec delay of the head node D^0_{MACsec}. When MKPDUs are high priority, MPDUs are low priority and experience increasing blocking, delay rises from 0.034s to 0.107s, a 214% increase.

l^0_{MACsec} **in bytes**	T_{WC} **in s (High Prio, Low Prio)**	D^0_{MACsec} **in s (High Prio, Low Prio)**
50	(0.448, 0.556)	(0.034, 0.01000)
356	(0.168, 0.256)	(0.015, 0.01020)
1200	(0.496, 2.640)	(0.076, 0.059)
1486	(0.610, 15.98)	(0.107, 0.085)

Table 3. Experiment 3: Impact of MKA's priority on the Worst-case time to final key agreement T_{WC} and worst-case delay of MPDUs D^0_{MACsec} in a 16-node network.

7 Discussion and Ongoing Standardization

Our work applies state-of-the-art Network Calculus to formally analyze and bound the worst-case timing behavior of key agreement protocols over 10BASE-T1S Ethernet with PLCA. One of NC's key strengths lies in its flexibility. It allows us to systematically vary parameters such as the number of nodes, packet sizes, or inter-arrival times to observe their impact on the overall system performance. We identified the worst-case message sequences for MKA and IKA to ensure the analysis reflects the actual protocol behavior; see Figure 7 and Figure 8. The analysis demonstrated that NC is well-suited for deriving worst-case guarantees on message delays under strict queuing assumptions and control- and data-plane competition in MKA's case.

One limitation of the paper is that we conservatively model a scenario where the head node (resp. Key Server) always experiences the worst-case delay $D^0_{\text{MKA/IKA}}$ for MKPDU or IPDU it sends. While this allows for a worst-case upper bound, it may overestimate real-world latencies. Additionally, we assume all nodes i continuously transmit maximum-sized packets, ensuring a fully loaded network. In practical deployments, nodes may be silent or only sporadically active. However, this conservatism can be reduced, as shown in related work on WRR scheduling [18]. Furthermore, we focus on deterministic Network Calculus and do not take the likelihood of worst-case scenarios into account. Our analysis identifies an existing guarantee on T_{WC} but does not quantify how often it may occur.

Overall, our findings demonstrate that Infineon's proposed IKA protocol emerges as a promising alternative to standard MKA, particularly in scenarios involving small-sized MPDUs, where MKA exhibits poor scalability and excessive worst-case delays. In fact, our analysis shows that IKA can guarantee key agreement within the allowed time span of 30ms for up to 11 nodes, provided the IPDU size is limited to 120+14 bytes. In contrast, MKA reaches 92.7 ms under the same conditions. Within the IEEE 802.1 working group, several optimizations to improve the efficiency of MKA in group-based secure communication are currently discussed (see recent Revision 2.2 of M. Seaman [17]). Revision 2.2 improves the average-case performance by reducing unnecessary computational overhead during key distribution and MKPDU validation (nevertheless, MKA's worst-case behavior remains unchanged). Because the proposed mechanism transmits key numbers in-line (indirectly using the port field of the SecTAG), it may be possible to define a hybrid approach by adopting the principles of IKA and where some nodes store key numbers in non-volatile memory, allowing them to recover very quickly from asynchronous reset events yet also can communicate securely with ordinary MKA nodes. This is a topic for future research.

References

[1] A. Bouillard. "Stability and Performance Bounds in Cyclic Networks Using Network Calculus". *Formal Modeling and Analysis of Timed Systems*. Ed. by É. André and M. Stoelinga. Springer International Publishing, 2019, pp. 96–113. ISBN: 978-3-030-29662-9.

[2] L. Chen. *Recommendation for Key Derivation Using Pseudorandom Functions*. Tech. rep. NIST Special Publication (SP) 800-108 Rev. 1. National Institute of Standards and Technology, Aug. 2022. DOI: 10.6028/NIST.SP.800-108r1.

[3] P. Decker. *Security concepts with CAN XL*. https://www.can-cia.org/fileadmin/cia/documents/proceedings/2024_decker.pdf. Accessed: 2025-03-30.

[4] M. Eaker. *Zone architecture, Ethernet drive vehicle of the future*. www.ti.com/lit/pdf/ssztd02. Accessed: 2025-03-30. 2023.

[5] "IEEE Standard for Ethernet". *IEEE Std 802.3-2022 (Revision of IEEE Std 802.3-2018)* (2022). DOI: 10.1109/IEEESTD.2022.9844436.

[6] "IEEE Standard for Ethernet - Amendment 5: Physical Layer Specifications and Management Parameters for 10 Mb/s Operation and Associated Power Delivery over a Single Balanced Pair of Conductors". *IEEE Std 802.3cg-2019 (Amendment to IEEE Std 802.3-2018 as amended by IEEE Std 802.3cb-2018, IEEE Std 802.3bt-2018, IEEE Std 802.3cd-2018, and IEEE Std 802.3cn-2019)* (2020). DOI: 10.1109/IEEESTD.2020.8982251.

[7] "IEEE Standard for Local and Metropolitan Area Networks–Port-Based Network Access Control". *IEEE Std 802.1X-2020 (Revision of IEEE Std 802.1X-2010 Incorporating IEEE Std 802.1Xbx-2014 and IEEE Std 802.1Xck-2018)* (2020).

[8] "IEEE Standard for Local and metropolitan area networks-Media Access Control (MAC) Security". *IEEE Std 802.1AE-2018 (Revision of IEEE Std 802.1AE-2006)* (2018). DOI: 10.1109/IEEESTD.2018.8585421.

[9] W. Kellerer and A. V. Bemten. *Network Calculus: A Comprehensive Guide*. Tech. rep. Chair of Communication Network, Technical University of Munich, 2016.

[10] J.-Y. Le Boudec and P. Thiran. *Network Calculus: A Theory of Deterministic Queuing Systems for the Internet*. 2004.

[11] L. Lo Bello, G. Patti, and L. Leonardi. "A Perspective on Ethernet in Automotive Communications—Current Status and Future Trends". *Applied Sciences* (2023). DOI: 10.3390/app13031278.

[12] K. Matheus and T. Königseder. *Automotive Ethernet*. Cambridge University Press, 2021.

[13] J. Min and Y. Park. "Performance Enhancement of In-Vehicle 10BASE-T1S Ethernet Using Node Prioritization and Packet Segmentation". *IEEE Access* 10 (2022), pp. 103286–103295. DOI: 10.1109/ACCESS.2022.3209824.

[14] D. A. Nascimento, S. Bondorf, and D. R. Campelo. "Modeling and Analysis of Time-Aware Shaper on Half-Duplex Ethernet PLCA Multidrop". *IEEE Transactions on Communications* (2023).

[15] Open Alliance. *Automotive MKA Specification*. https://opensig.org/wp-content/uploads/2025/03/OA_MACsec_Automotive-MKA-v1.pdf. Accessed: 2025-03-30.

[16] Open Alliance. *MACsec Automotive Profile*. https://opensig.org/tech-committee/tc17-macsec-automotive-profile/. Accessed: 2025-03-30.

[17] M. Seaman. *MKA optimization for group CAs, Revision 2.2*. https://www.ieee802.org/1/files/public/docs2025/x-seaman-mka-optimizations-0325-v06.pdf. Accessed: 2025-03-30. 2025.

[18] A. Soni et al. "WCTT analysis of avionics Switched Ethernet Network with WRR Scheduling". *Proceedings of the 26th International Conference on Real-Time Networks and Systems*. RTNS '18. Chasseneuil-du-Poitou, France: Association for Computing Machinery, 2018, pp. 213–222. ISBN: 9781450364638. DOI: 10.1145/3273905.3273925.

[19] L. Völker. *MACsec and beyond: the upcoming Network Security State of the Art*. https://automotive-macsec.com/papers.shtml. Accessed: 2025-03-30. 2024.

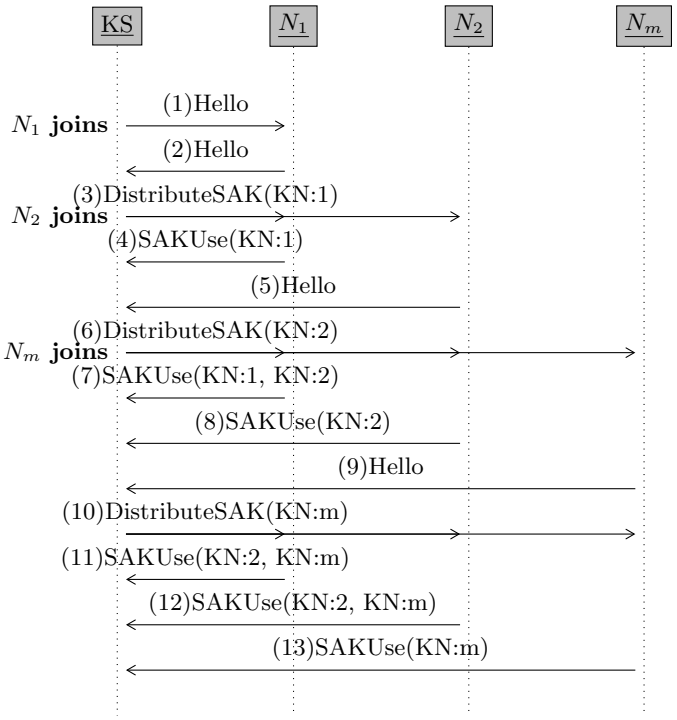

Fig. 7. Worst-case MKA SAK distribution in a network with one Key Server (KS) and m nodes $N_1, N_2, \ldots, N_m$. The nodes join sequentially and cause a new SAK distribution.

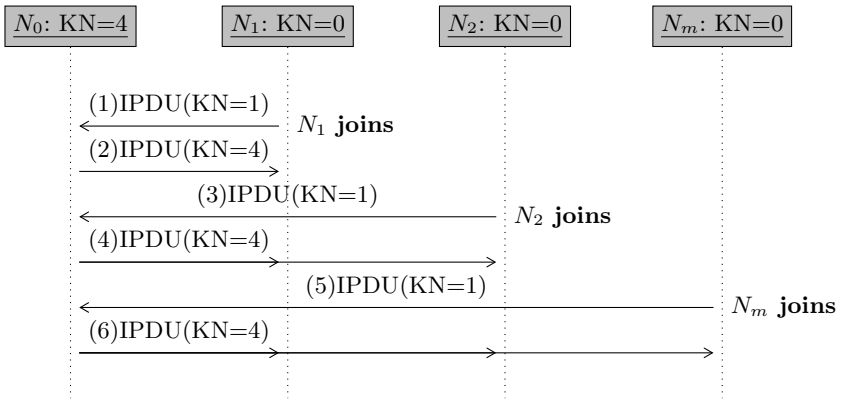

Fig. 8. Worst-case IKA SAK distribution in a network with one head node N_0 and m nodes $N_1, N_2, \ldots, N_m$, each storing its own Key Number (KN). Re-key is necessary due to KN mismatch.